
Object-Oriented Programming in C++

Complete Learning Notes
Basics to Advanced — 10-Day Roadmap

Topics Covered:

Basics • Classes & Objects • Constructors & Destructors
Encapsulation • Inheritance • Static Members
Polymorphism • Abstraction • Exception Handling
File Handling • OOP Implementation

Prakash Palsaniya

GitHub: [PrakashPalsaniya/OOPS-C-](https://github.com/PrakashPalsaniya/OOPS-C-)

June 20, 2026

Contents

1	C++ Basics Refresher	3
1.1	Variables and Data Types	3
1.2	Pointers and References	3
1.3	Functions	4
1.4	Stack vs Heap Memory	4
2	Classes and Objects	6
2.1	What is a Class?	6
2.2	Access Specifiers	6
2.3	The <code>this</code> Pointer	7
3	Constructors and Destructors	8
3.1	What is a Constructor?	8
3.2	Types of Constructors	8
3.2.1	Default Constructor	8
3.2.2	Parameterized Constructor	8
3.2.3	Copy Constructor	8
3.2.4	Constructor Overloading	9
3.2.5	Member Initializer List (Inline Constructor)	9
3.3	Destructor	9
4	Encapsulation	10
4.1	What is Encapsulation?	10
4.2	Getters and Setters	10
4.3	Benefits of Encapsulation	11
5	Inheritance	12
5.1	What is Inheritance?	12
5.2	Access in Inheritance	12
6	Types of Inheritance	13
6.1	Single Inheritance	13
6.2	Multilevel Inheritance	13
6.3	Multiple Inheritance	13
6.4	Hierarchical Inheritance	13
6.5	Hybrid Inheritance	14
7	Static Data Members and Functions	15
7.1	Static Data Members	15
7.2	Static Member Functions	15

8	Polymorphism	17
8.1	What is Polymorphism?	17
8.2	Function Overloading (Compile-time)	17
8.3	Operator Overloading (Compile-time)	17
8.4	Virtual Functions (Runtime Polymorphism)	18
9	Abstraction	20
9.1	What is Abstraction?	20
9.2	Abstract Class and Pure Virtual Functions	20
10	Exception Handling	22
10.1	What is Exception Handling?	22
10.2	Multiple Catch Blocks	22
10.3	Custom Exception Class	22
11	File Handling	24
11.1	File Handling in C++	24
12	OOP Implementation	26
12.1	Combining All OOP Concepts	26
12.2	OOP Quick Revision Cheat Sheet	28

Chapter 1

C++ Basics Refresher

1.1 Variables and Data Types

A **variable** is a named memory location that stores a value. Every variable has a **data type** which determines what kind of value it holds.

Data Type	Size	Example	Use
int	4 bytes	int age = 22;	Whole numbers
float	4 bytes	float x = 3.14;	Decimal (low precision)
double	8 bytes	double y = 3.14;	Decimal (high precision)
char	1 byte	char c = 'A';	Single character
bool	1 byte	bool b = true;	true / false
string	variable	string s = "hi";	Text

```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int    age    = 22;
6      float  marks  = 89.5;
7      double pi     = 3.14159265;
8      char   grade  = 'A';
9      bool   passed = true;
10     string name   = "Prakash";
11
12     cout << "Name : " << name << endl;
13     cout << "Age  : " << age  << endl;
14     cout << "Marks : " << marks << endl;
15     cout << "Grade : " << grade << endl;
16     return 0;
17 }
```

Listing 1.1: Variables and Data Types

1.2 Pointers and References

Pointer: A variable that stores the *memory address* of another variable.

Reference: An *alias* (another name) for an existing variable.

```

1  int num = 10;
2
3  int *ptr = &num;    // pointer: stores address of num
4  int &ref = num;     // reference: alias of num
5
6  cout << ptr;        // prints address
7  cout << *ptr;       // dereference: prints 10
8  *ptr = 50;          // changes num to 50
9  cout << ref;        // also prints 50 (same variable)

```

Listing 1.2: Pointers and References

Note

***ptr** means “value at address stored in ptr” (dereferencing).
&num means “address of num”.

1.3 Functions

```

1  // Call by value -- original NOT changed
2  void addValue(int x){ x += 10; }
3
4  // Call by reference -- original IS changed
5  void addRef(int &x){ x += 10; }
6
7  // Call by pointer -- original IS changed
8  void addPtr(int *x){ *x += 10; }
9
10 int main(){
11     int a = 5;
12     addValue(a);  cout << a;  // 5   (unchanged)
13     addRef(a);    cout << a;  // 15  (changed)
14     addPtr(&a);   cout << a;  // 25  (changed)
15 }

```

Listing 1.3: Functions – Call by Value, Reference, Pointer

1.4 Stack vs Heap Memory

Feature	Stack	Heap
Management	Automatic	Manual (new / delete)
Speed	Fast	Slower
Size	Limited	Large
Lifetime	Function scope	Until delete is called
Example	int x = 5;	int *p = new int;

```

1  // Stack -- automatic
2  int a = 10;

```

```
3
4 // Heap -- manual
5 int *ptr = new int;    // allocate
6 *ptr = 99;
7 delete ptr;           // free
8 ptr = nullptr;        // good practice
9
10 // Heap array
11 int *arr = new int[5];
12 delete[] arr;         // use delete[] for arrays
```

Listing 1.4: Stack vs Heap

Important

Always delete heap memory after use to avoid **memory leaks**.
Use `delete ptr;` for single variable, `delete[] arr;` for arrays.

Chapter 2

Classes and Objects

2.1 What is a Class?

A **class** is a user-defined data type that acts as a *blueprint* for creating objects. It groups **data members** (variables) and **member functions** (methods) together.

```
1  class Student {
2      public:
3          string name;           // data member
4          int    age;
5          int    rollNumber;
6
7          void display(){        // member function
8              cout << name << " | " << age << endl;
9          }
10 };
11
12 int main(){
13     Student S1;                // creating an object
14     S1.name = "Prakash";
15     S1.age  = 20;
16     S1.rollNumber = 101;
17     S1.display();
18 }
```

Listing 2.1: Basic Class and Object

2.2 Access Specifiers

Specifier	Same Class	Child Class	Outside
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

```
1  class BankAccount {
2      private:
3          int balance;           // only accessible inside class
4
5      protected:
6          string accountType;    // accessible in child class too
7
8      public:
9          string ownerName;      // accessible from anywhere
10         void setBalance(int amount){ balance = amount; }
```

```
11     void display(){ cout << ownerName << " " << balance; }  
12 };
```

Listing 2.2: Access Specifiers

2.3 The this Pointer

this is a special pointer inside every class that points to the **current object**. It resolves conflicts when parameter names match data member names.

```
1  class Student {  
2      string name;  
3      int age;  
4  public:  
5      Student(string name, int age){  
6          this->name = name;    // this->name = member  
7          this->age  = age;      //          name = parameter  
8      }  
9      // Returning current object for method chaining  
10     Student& setName(string name){  
11         this->name = name;  
12         return *this;  
13     }  
14 };
```

Listing 2.3: this pointer

Chapter 3

Constructors and Destructors

3.1 What is a Constructor?

A **constructor** is a special member function called *automatically* when an object is created.

- Same name as the class
- No return type (not even void)
- Used to initialize data members

3.2 Types of Constructors

3.2.1 Default Constructor

Called when no arguments are passed.

```
1 Customer(){
2     name      = "Default";
3     balance = 0;
4 }
5 Customer A1;    // default constructor called
```

3.2.2 Parameterized Constructor

Accepts arguments to initialize with custom values.

```
1 Customer(string n, int ac, int bal){
2     this->name      = n;
3     this->ac         = ac;
4     this->balance = bal;
5 }
6 Customer A1("Prakash", 101, 5000);
```

3.2.3 Copy Constructor

Creates a new object as a copy of an existing object.

```
1 Customer(Customer &obj){
2     name      = obj.name;
3     balance = obj.balance;
4 }
5 Customer A2(A1);    // copy constructor called
```

3.2.4 Constructor Overloading

Multiple constructors with different parameters in the same class.

```
1 Customer(){} // 0 args
2 Customer(string n, int b, int c){} // 3 args
3 Customer(string n, int b){} // 2 args
4 // Compiler picks correct one based on call
```

3.2.5 Member Initializer List (Inline Constructor)

More efficient – initializes members directly instead of assigning inside body.

```
1 Customer(string a, int b, int c)
2 : name(a), ac(b), balance(c) {}
```

3.3 Destructor

A **destructor** is called automatically when an object goes out of scope or is deleted.

```
1 class Customer {
2     int *data;
3     public:
4     Customer(){
5         data = new int; // allocate on heap
6         *data = 10;
7         cout << "Constructor called" << endl;
8     }
9     ~Customer(){ // destructor
10        delete data; // free heap memory
11        cout << "Destructor called" << endl;
12    }
13 };
14 int main(){ Customer A1; } // destructor auto-called at end
```

Listing 3.1: Destructor – free dynamic memory

Note

Call Order: Constructors are called in the order objects are created. Destructors are called in *reverse* order (LIFO).

Chapter 4

Encapsulation

4.1 What is Encapsulation?

Encapsulation is bundling of data and functions into a single unit (class), while restricting direct access to the data from outside.

A **capsule** hides medicine inside. Similarly, a class hides its private data and exposes only safe public methods.

4.2 Getters and Setters

```
1  class Student {
2      private:
3          string name;
4          int age;
5          float marks;
6
7      public:
8          // Setter -- with validation
9          void setAge(int age){
10             if(age > 0 && age < 100)
11                 this->age = age;
12             else
13                 cout << "Invalid age!" << endl;
14         }
15         // Getter
16         int getAge(){ return age; }
17
18         void setMarks(float m){
19             if(m >= 0 && m <= 100)
20                 marks = m;
21             else
22                 cout << "Marks must be 0-100!" << endl;
23         }
24         float getMarks(){ return marks; }
25     };
```

Listing 4.1: Encapsulation with getter and setter

4.3 Benefits of Encapsulation

- **Data hiding:** Private data cannot be accessed directly
- **Validation:** Setters validate data before storing
- **Flexibility:** Internal implementation can change without affecting outside code
- **Security:** Prevents accidental modification of data

Tip

Always keep data `private` and provide `public` getter/setter methods. This gives you full control over how data is read and modified.

Chapter 5

Inheritance

5.1 What is Inheritance?

Inheritance allows a class (child/derived) to acquire properties and behaviors of another class (parent/base). It promotes **code reuse**.

```
1 // Base class (Parent)
2 class Animal {
3     public:
4         string name;
5         void eat() { cout << "Animal is eating" << endl; }
6         void sleep(){ cout << "Animal is sleeping" << endl; }
7 };
8
9 // Derived class (Child) -- gets all public members of Animal
10 class Dog : public Animal {
11     public:
12         void bark(){ cout << "Dog is barking" << endl; }
13 };
14
15 int main(){
16     Dog d;
17     d.name = "Bruno";
18     d.eat();    // inherited from Animal
19     d.bark();   // own method
20 }
```

Listing 5.1: Basic Inheritance

5.2 Access in Inheritance

Member in Base	public inh.	protected inh.	private inh.
public	public	protected	private
protected	protected	protected	private
private	not inherited	not inherited	not inherited

Chapter 6

Types of Inheritance

6.1 Single Inheritance

One child inherits from one parent.

```
1 class A { public: void showA(){} };
2 class B : public A {                      // B inherits from A
3     public: void showB(){}
4 };;
```

6.2 Multilevel Inheritance

Chain: A inherits nothing, B inherits A, C inherits B.

```
1 class Animal { public: void eat(){} };
2 class Dog : public Animal { public: void bark(){} };
3 class BabyDog: public Dog { public: void weep(){} };
4
5 BabyDog bd;
6 bd.eat();    // from Animal (Level 1)
7 bd.bark();   // from Dog (Level 2)
8 bd.weep();   // own method (Level 3)
```

6.3 Multiple Inheritance

One child inherits from more than one parent.

```
1 class Father { public: void property(){} };
2 class Mother { public: void cooking(){} };
3
4 // Child gets both Father's and Mother's methods
5 class Child : public Father, public Mother {
6     public: void study(){}
7 };
```

Important

Diamond Problem: When two parents share the same function, multiple inheritance causes ambiguity. Solved using virtual inheritance.

6.4 Hierarchical Inheritance

Multiple children inherit from ONE parent.

```
1 class Animal { public: void eat(){} };
2
3 class Dog : public Animal { public: void bark(){} };
4 class Cat : public Animal { public: void meow(){} };
5 class Cow : public Animal { public: void moo(){} };
```

6.5 Hybrid Inheritance

Combination of two or more types of inheritance.

```
1 // Example: Hierarchical + Multiple
2 class Vehicle { };
3 class Car : public Vehicle { };
4 class Boat : public Vehicle { };
5 class AmphibiousCar : public Car, public Boat { };
```

Chapter 7

Static Data Members and Functions

7.1 Static Data Members

A **static data member** is *shared across all objects* of a class. Only **one copy** exists in memory regardless of how many objects are created.

```
1 class Customer {
2     string name;
3     static int total_customer;    // ONE copy shared by ALL
                                   // objects
4 public:
5     Customer(string n, int ac, int bal){
6         this->name = n;
7         total_customer++;        // increments for every new
                                   // object
8     }
9     void display(){
10        cout << name << " | Total: " << total_customer << endl;
11    }
12 };
13
14 // MUST initialize static member outside class
15 int Customer::total_customer = 0;
16
17 int main(){
18     Customer A1("Prakash", 1, 1000); // total = 1
19     Customer A2("Rohit", 2, 2000);   // total = 2
20     A1.display(); // prints total = 2
21 }
```

Listing 7.1: Static data member

7.2 Static Member Functions

Static member functions can be called *without creating an object*. They can only access *static data members*.

```
1 class Customer {
2     static int total;
3     static int totalBalance;
4 public:
5     // Static function -- called using class name, no object
                                   // needed
6     static void showStats(){
7         cout << "Total Customers: " << total << endl;
```



```

8         cout << "Total Balance : " << totalBalance << endl;
9     }
10 };
11
12 // Call WITHOUT creating any object
13 Customer::showStats();

```

Listing 7.2: Static member function

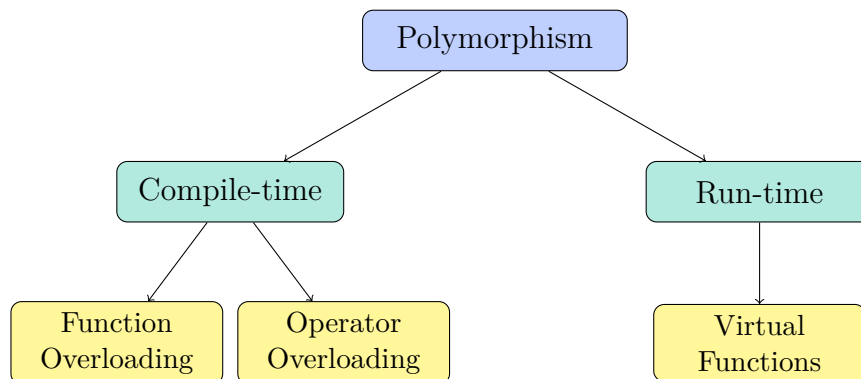
Feature	Static Member	Normal Member
Shared across all objects	Yes	No
Called without object	Yes (static func.)	No
Can access non-static members	No	Yes

Chapter 8

Polymorphism

8.1 What is Polymorphism?

Polymorphism means “many forms”. The same function name behaves differently based on context.



8.2 Function Overloading (Compile-time)

Same function name, different parameters. Compiler picks correct version at compile time.

```
1 int    add(int a, int b)      { return a + b; }
2 int    add(int a, int b, int c) { return a + b + c; }
3 double add(double a, double b) { return a + b; }
4
5 add(10, 20);      // calls 1st
6 add(10, 20, 30);  // calls 2nd
7 add(1.5, 2.5);    // calls 3rd
```

Listing 8.1: Function Overloading

8.3 Operator Overloading (Compile-time)

Redefine how operators work for user-defined classes.

```
1 class Complex {
2     public:
3         int real, imag;
4         Complex(int r, int i): real(r), imag(i){}
5
6         // Overloading + operator
7         Complex operator+(Complex c){
8             return Complex(real + c.real, imag + c.imag);
9         }
```

```

9     }
10    void display(){ cout << real << "+" << imag << "i"; }
11 };
12
13 Complex c1(3,4), c2(1,2);
14 Complex c3 = c1 + c2;    // uses overloaded +
15 c3.display();           // prints 4+6i
    
```

Listing 8.2: Operator Overloading – Complex numbers

8.4 Virtual Functions (Runtime Polymorphism)

The correct version is selected at *runtime* based on the actual object type.

```

1  class Animal {
2      public:
3          virtual void sound(){           // virtual keyword
4              cout << "Some sound" << endl;
5          }
6  };
7
8  class Dog : public Animal {
9      public:
10         void sound() override {         // override keyword
11             cout << "Woof!" << endl;
12         }
13 };
14
15 class Cat : public Animal {
16     public:
17         void sound() override {
18             cout << "Meow!" << endl;
19         }
20 };
21
22 int main(){
23     Animal *ptr;
24
25     Dog d; ptr = &d;
26     ptr->sound();    // calls Dog's sound() -- decided at
                       runtime
27
28     Cat c; ptr = &c;
29     ptr->sound();    // calls Cat's sound() -- decided at
                       runtime
30 }
    
```

Listing 8.3: Virtual Functions and Runtime Polymorphism

Tip

Always use **override** in the child class. If the base function signature does not match, the compiler will give an error – helping you catch bugs early.

Chapter 9

Abstraction

9.1 What is Abstraction?

Abstraction means showing only *what is necessary* and hiding internal implementation details.

Example: A car driver uses the steering wheel and pedals without knowing the internal engine mechanics.

9.2 Abstract Class and Pure Virtual Functions

A class with at least one **pure virtual function** is an **abstract class**. You *cannot* create objects of an abstract class.

```
1  class Shape {                                // Abstract class
2      public:
3          virtual float area()                  = 0;    // pure virtual (= 0)
4          virtual float perimeter()             = 0;    // pure virtual
5
6          void display(){                       // regular function
7              cout << "Area      : " << area()        << endl;
8              cout << "Perimeter: " << perimeter()    << endl;
9          }
10 };
11
12 class Circle : public Shape {
13     float r;
14     public:
15     Circle(float r){ this->r = r; }
16     float area()      override { return 3.14f * r * r; }
17     float perimeter() override { return 2 * 3.14f * r; }
18 };
19
20 class Rectangle : public Shape {
21     float l, w;
22     public:
23     Rectangle(float l, float w): l(l), w(w){}
24     float area()      override { return l * w; }
25     float perimeter() override { return 2*(l+w); }
26 };
27
28 int main(){
29     // Shape s;    ERROR -- cannot instantiate abstract class
30     Circle c(7);
31     c.display();
```

```
32
33     Rectangle r(5,3);
34     r.display();
35 }
```

Listing 9.1: Pure Virtual Function and Abstract Class

Encapsulation	Hides data using private access
Abstraction	Hides implementation using pure virtual functions

Chapter 10

Exception Handling

10.1 What is Exception Handling?

Exception handling manages runtime errors gracefully – preventing program crashes.

Keyword	Purpose
try	Block of code that might throw an exception
throw	Raises / throws an exception
catch	Catches and handles the exception

```
1 int divide(int a, int b){
2     if(b == 0)
3         throw "Division by zero!";    // throw an exception
4     return a / b;
5 }
6
7 int main(){
8     try {
9         int result = divide(10, 0);    // might throw
10        cout << result;
11    }
12    catch(const char *msg){             // catch the exception
13        cout << "Error: " << msg << endl;
14    }
15    cout << "Program continues..." << endl;
16 }
```

Listing 10.1: Basic Exception Handling

10.2 Multiple Catch Blocks

```
1 try {
2     // code that might throw
3 }
4 catch(int e)          { cout << "Int exception: " << e; }
5 catch(const char *e){ cout << "String exception: " << e; }
6 catch(...)           { cout << "Unknown exception caught"; }
```

Listing 10.2: Multiple catch blocks

10.3 Custom Exception Class

```
1 class InvalidAgeException : public exception {
2     public:
3         const char* what() const noexcept override {
4             return "Invalid Age! Must be between 0-120.";
5         }
6 };
7
8 void setAge(int age){
9     if(age < 0 || age > 120)
10        throw InvalidAgeException();
11 }
12
13 int main(){
14     try { setAge(-5); }
15     catch(InvalidAgeException &e){
16         cout << "Caught: " << e.what() << endl;
17     }
18 }
```

Listing 10.3: Custom Exception

Chapter 11

File Handling

11.1 File Handling in C++

File handling lets us read from and write to files persistently. Requires `#include<fstream>`.

Class	Purpose
<code>ofstream</code>	Write to file (output)
<code>ifstream</code>	Read from file (input)
<code>fstream</code>	Both read and write

```
1 #include<fstream>
2 using namespace std;
3
4 int main(){
5     ofstream file("data.txt");           // open for writing
6     if(file.is_open()){
7         file << "Hello, File!" << endl;
8         file << "OOP in C++" << endl;
9         file.close();                     // always close
10    }
11 }
```

Listing 11.1: Write to a file

```
1 #include<fstream>
2 using namespace std;
3
4 int main(){
5     ifstream file("data.txt");           // open for reading
6     string line;
7     while(getline(file, line)){          // read line by line
8         cout << line << endl;
9     }
10    file.close();
11 }
```

Listing 11.2: Read from a file

```
1 class Student {
2     public:
3         string name;
4         int age;
5
6         void saveToFile(){
7             ofstream f("student.txt");
```

```
8         f << name << " " << age;
9         f.close();
10    }
11    void loadFromFile(){
12        ifstream f("student.txt");
13        f >> name >> age;
14        f.close();
15    }
16    void display(){ cout << name << " " << age << endl; }
17 };
18
19 int main(){
20     Student s1; s1.name = "Prakash"; s1.age = 20;
21     s1.saveToFile();
22
23     Student s2;
24     s2.loadFromFile();
25     s2.display();           // prints Prakash 20
26 }
```

Listing 11.3: File Handling with OOP

Chapter 12

OOP Implementation

12.1 Combining All OOP Concepts

A well-designed OOP program uses all four pillars together:

1. **Encapsulation** — keep data private, expose via public methods
2. **Abstraction** — hide implementation, show only interface
3. **Inheritance** — reuse existing code in child classes
4. **Polymorphism** — one interface, many behaviors

```
1  #include <iostream>
2  using namespace std;
3
4  // Abstract Base Class (Abstraction)
5  class Account {
6      protected:                                // protected for child access
7          string name;
8          int accNo;
9          double balance;
10
11     public:
12         Account(string n, int a, double b)
13             : name(n), accNo(a), balance(b){}
14
15         virtual void display() = 0; // pure virtual (Abstraction)
16
17         // Encapsulated operations with validation
18         void deposit(double amt){
19             if(amt > 0) balance += amt;
20         }
21         void withdraw(double amt){
22             if(amt > 0 && amt <= balance) balance -= amt;
23             else cout << "Insufficient balance!" << endl;
24         }
25 };
26
27 // Inheritance -- SavingsAccount extends Account
28 class SavingsAccount : public Account {
29     double interestRate;
30     public:
31         SavingsAccount(string n, int a, double b, double r)
32             : Account(n, a, b), interestRate(r){}
```

```
33
34     // Runtime Polymorphism -- overrides display()
35     void display() override {
36         cout << "Name      : " << name          << endl;
37         cout << "Acc No   : " << accNo          << endl;
38         cout << "Balance  : " << balance         << endl;
39         cout << "Rate     : " << interestRate << "%" << endl;
40     }
41 };
42
43 int main(){
44     Account *acc = new SavingsAccount("Prakash",101,5000,4.5);
45     acc->deposit(1000);
46     acc->display();    // calls SavingsAccount::display()
47     delete acc;
48 }
```

Listing 12.1: Mini Bank System – all OOP concepts

12.2 OOP Quick Revision Cheat Sheet

OOP in C++ – Cheat Sheet

class	Blueprint for objects
object	Instance of a class
constructor	Auto-called when object is created
destructor	Auto-called when object is destroyed
this	Pointer to the current object
static	Shared across all objects
private	Hidden from outside (Encapsulation)
protected	Accessible in derived class
virtual	Enables runtime polymorphism
override	Explicitly overrides base function
= 0	Pure virtual – makes class abstract
try/catch/throw	Exception handling
ofstream	Write to file
ifstream	Read from file
new	Allocate heap memory
delete	Free heap memory

4 Pillars of OOP

Encapsulation	Wrap data + functions. Keep data private .
Abstraction	Show only essentials. Use pure virtual functions.
Inheritance	Reuse code. Child inherits from parent.
Polymorphism	One interface, many forms. Overloading + Virtual.

Keep Learning. Keep Coding.

github.com/PrakashPalsaniya/OOPS-C-